DevOps - guia de estudo e preparação

Introdução ao Kubernetes

Para que serve, afinal?

Imagine que uma empresa mantém uma loja virtual. Há uma aplicação que recebe pedidos, outra que calcula o frete e outra que confirma pagamentos. Cada uma precisa ser iniciada, configurada, atualizada e acompanhada. Se um processo parar ou o movimento aumentar, alguém terá de tomar providências para que o atendimento continue.

Em uma instalação pequena, essas tarefas podem ser feitas diretamente no servidor. À medida que aumentam o número de aplicações, de máquinas e de entregas, administrar tudo manualmente fica mais trabalhoso. Kubernetes oferece uma maneira de organizar a execução de aplicações em contêineres. A equipe informa como elas devem funcionar e o sistema procura manter essa configuração: quais imagens executar, quantas cópias manter e quais instâncias estão prontas para atender. [1–5]

Isso aparece, por exemplo, na publicação de APIs, em serviços de comércio eletrônico e em plataformas com vários serviços que precisam ser atualizados separadamente. É o mesmo tipo de problema que estudaremos aqui, em escala menor: publicar uma aplicação, manter suas cópias funcionando, alterar a versão e recuperar a anterior quando a entrega falhar.

DevOps reúne práticas de desenvolvimento e operação para entregar software com colaboração, automação e acompanhamento. Em vez de tratar a publicação como uma tarefa separada do desenvolvimento, a equipe acompanha o resultado da entrega e utiliza o que observa para corrigir e melhorar o processo. Kubernetes é uma das ferramentas que podem participar desse trabalho. [17]

Onde esse conhecimento é usado

Kubernetes pode operar em servidores da própria organização ou em nuvem. AWS, Microsoft Azure e Google Cloud oferecem serviços gerenciados baseados nele: EKS, AKS e GKE. Nesses ambientes, parte da administração do cluster fica com o provedor; a equipe ainda precisa conhecer a aplicação e saber publicá-la e acompanhar seu funcionamento. [16]

Para quem desenvolve software, conhecer esse processo ajuda a entender o que acontece depois que o código é escrito. Você passa a interpretar uma falha de inicialização, uma configuração incorreta e uma atualização que não conclui. Também consegue conversar com a equipe que opera o ambiente usando os mesmos conceitos. Essa é uma vantagem concreta em trabalhos de backend, DevOps, infraestrutura e engenharia de plataforma: além de programar, você compreende como o software chega ao usuário e permanece disponível.

Essa formação complementa o conhecimento de programação com experiência de publicação e diagnóstico. E Kubernetes não é obrigatório em todo projeto. Para uma aplicação pequena, uma solução mais simples pode atender melhor. Aprendê-lo também serve para reconhecer quando seu uso faz sentido.

Neste material. Usaremos Minikube para criar um cluster no próprio computador. Não haverá contratação de nuvem nem publicação em repositório remoto. A instalação deve ser feita antes da atividade.

1 | Antes do Kubernetes

Programa, imagem e contêiner

Um **processo** é um programa em execução. Quando você inicia um servidor Python, o sistema operacional cria um processo que utiliza memória e CPU. O arquivo com o código, sozinho, ainda não é esse processo.

Para executar a aplicação em outro computador, precisamos levar também o ambiente de que ela depende. Uma **imagem de contêiner** reúne os arquivos e as instruções que servem de base para essa execução. No exercício, ela contém Python e o arquivo da aplicação. O **contêiner** é uma execução dessa imagem, com isolamento de processos e recursos oferecido pelo sistema. Contêiner não é sinônimo de máquina virtual. [11]

Pense no instalador de um programa e no programa já aberto: um é o pacote; o outro é a execução. A comparação não descreve todos os detalhes técnicos, mas ajuda a separar imagem e contêiner. Podemos iniciar várias execuções a partir da mesma imagem.

O **Docker** será usado para construir as imagens. O **Dockerfile** é o arquivo que descreve essa construção. Observe o arquivo fornecido:

```
FROM python:3.12-slim
ARG APP_VERSION=v1
ENV APP_VERSION=${APP_VERSION} \
    PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PORT=8000
WORKDIR /app
COPY app/main.py /app/main.py
USER 10001:10001
EXPOSE 8000
CMD ["python", "/app/main.py"]
```

`FROM` escolhe a imagem-base; `COPY` leva o código para a imagem; `CMD` define o comando de início. `USER` evita executar a aplicação como root. `EXPOSE` documenta a porta do contêiner; não abre sozinho uma porta no computador.

A identificação `entrega-local:v1` contém um nome e uma **tag**, neste caso `v1`. Neste exercício, cada entrega terá uma tag diferente. O argumento `APP_VERSION` também deve receber a mesma versão, pois é ele que define o valor mostrado na página.

Sem banco de dados. A aplicação do laboratório apenas responde a consultas. Ela não guarda pedidos ou cadastros. Chamamos isso de aplicação *sem estado* neste contexto: suas instâncias podem ser substituídas sem precisar recuperar dados produzidos por elas.

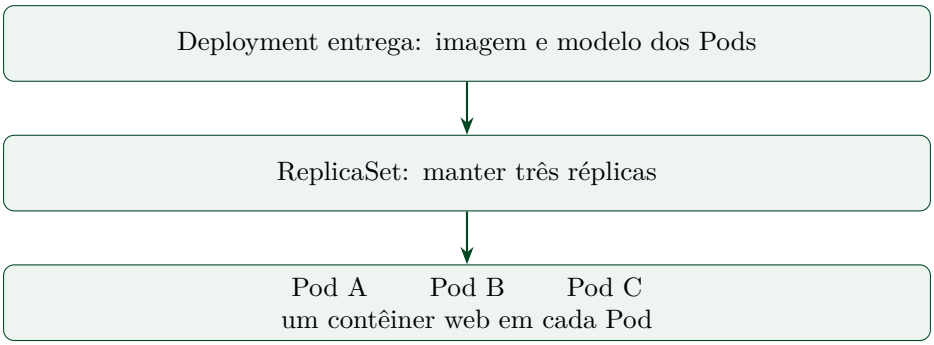
2 | A organização do ambiente

Cluster, nó, Pod e Deployment

Um **cluster** é o ambiente Kubernetes formado pelos componentes de controle e pelas máquinas que executam as aplicações. Essas máquinas são chamadas de **nós** (*Nodes*). O **plano de controle** recebe as configurações, acompanha os objetos e coordena decisões, como escolher onde iniciar uma aplicação. [1]

No Kubernetes, a unidade que executa um ou mais contêineres é o **Pod**. Os contêineres de um mesmo Pod compartilham aspectos do ambiente, como a rede. Na prática desta aula, cada Pod terá apenas um contêiner, com nosso servidor web. [2]

Não vamos criar e cuidar de cada Pod individualmente. Um **Deployment** declara o modelo dos Pods e coordena sua substituição quando mudamos esse modelo. Ele gerencia um **ReplicaSet**, responsável por manter a quantidade solicitada. Cada cópia do modelo em execução é uma **réplica**. [3]



Se declaramos três réplicas e um Pod é excluído, o controlador procura criar outro para voltar à quantidade solicitada. Esse acompanhamento é chamado de **reconciliação**: comparar o *estado atual* com o *estado desejado* e agir sobre a diferença.

Um **namespace** organiza objetos dentro do cluster. Usaremos `aula-k8s` para agrupar os recursos do exercício. Já o **contexto** indica a configuração de acesso ao cluster usada pela ferramenta de comando; nosso contexto será `devops-k8s`.

> O Minikube desta aula terá um único nó. Três Pods nessa máquina permitem observar a substituição de uma instância, mas não protegem contra a perda da máquina inteira. Réplicas também não substituem backup.

Minikube é a ferramenta que cria o cluster local para estudo. **kubectl** é o programa de linha de comando que envia solicitações à API do Kubernetes. O Minikube consegue executar uma versão compatível do kubectl, por isso não o instalaremos separadamente. [9]

3 | Descrever e acessar a aplicação

Manifestos, Service e configuração

Um **manifesto** descreve um objeto do cluster em um arquivo. As chaves `apiVersion` e `kind` indicam a versão da API e o tipo do objeto; `metadata` reúne sua identificação; `spec` descreve a configuração desejada. Usaremos **YAML**, formato que organiza informações por chaves e indentação. Um exemplo de informação é `replicas: 3`: queremos três cópias do modelo. A indentação precisa ser preservada; use espaços em vez de tabulações.

Ao executar `apply -f arquivo.yaml`, enviamos essa descrição à API. O comando não significa que tudo já esteja pronto: o sistema ainda precisa criar ou atualizar os objetos. Por isso consultamos os Pods e aguardamos a atualização terminar.

As **labels** são etiquetas, como `app: entrega`. Um **selector** indica quais etiquetas procurar. O Service do exercício seleciona os Pods com essa label. O **Service** oferece um ponto de acesso estável, mesmo quando os Pods são substituídos e seus nomes ou endereços mudam. [4]

Uma **porta** identifica um ponto de comunicação de rede. Na aplicação, HTTP usa a porta 8000 do contêiner. **HTTP** é o protocolo usado pelo navegador para fazer pedidos e receber respostas. Uma **API** é uma interface para um programa utilizar funções ou consultar dados de outro; nossa API responde em `/api/status`, no formato **JSON**, composto por campos e valores.

Porta	Uso no exercício
8000	Servidor web dentro do contêiner.
80	Porta do Service dentro do cluster.
30080	NodePort: publicação do Service em uma porta dos nós.
8080	Acesso local aberto pelo aluno com port-forward.
8081	Inspeção direta do Pod com defeito.
18080	Acesso temporário aberto pela pipeline.

Usaremos **port-forward** para encaminhar uma porta do computador até um Pod selecionado. Assim, o navegador consegue acessar a aplicação sem depender do alcance do NodePort na rede do driver. Esse encaminhamento escolhe um Pod; repetir pedidos pela mesma sessão não é uma demonstração de balanceamento entre réplicas.

O **ConfigMap** armazena configurações não sigilosas separadas da imagem. Nele ficará a mensagem da página. Uma **variável de ambiente** é um valor recebido pelo processo quando ele inicia; usaremos esse mecanismo para ler a mensagem. Atualizar o ConfigMap não modifica as variáveis dos processos já iniciados. [5]

4 | Verificar e atualizar

Prontidão, rollout e rollback

Um servidor pode estar aberto e, mesmo assim, não conseguir atender corretamente. Por exemplo, ele iniciou, mas uma configuração necessária está inválida. As **probes** são verificações que o Kubernetes faz para acompanhar essas situações. [6]

Verificação	O que ela procura saber
Startup probe	A aplicação conseguiu iniciar? Enquanto essa verificação não passa, as outras aguardam.
Liveness probe	O contêiner continua saudável ou precisa ser reiniciado?
Readiness probe	A instância está pronta para receber tráfego?

Na versão defeituosa do exercício, `/healthz` responde HTTP 200 e `/readyz` responde HTTP 503. O código 200 indica sucesso nessa resposta; 503 indica indisponibilidade. O processo continua vivo, mas não fica pronto. A coluna READY pode mostrar `0/1`: nenhum dos seus contêineres está pronto. Não significa, por si só, que o processo parou.

Chamamos de **rollout** a aplicação de uma nova configuração do modelo dos Pods. Em uma **rolling update**, as instâncias são substituídas gradualmente. Nosso Deployment permite uma réplica extra e não permite reduzir a quantidade disponível durante essa substituição. Isso exige recursos livres no nó.

Uma **revisão** registra uma versão do modelo dos Pods no Deployment. A tag da imagem e a revisão do Deployment têm funções diferentes: modificar o modelo pode criar uma revisão mesmo sem mudar a imagem. O **rollback** volta a uma revisão anterior desse modelo; não desfaz automaticamente mudanças no ConfigMap ou nos arquivos do projeto. [3]

> Se a espera por uma atualização termina com timeout, o comando parou de esperar. O Kubernetes não executa rollback automaticamente por causa desse timeout. Primeiro investigamos; depois solicitamos a reversão.

Uma **pipeline** é uma sequência de tarefas automatizadas. A nossa executará testes, construção, publicação e uma consulta HTTP. Um **smoke test** é uma verificação curta de funcionamento básico; aqui, ele confere versão e prontidão. A **suíte de testes** é o conjunto de testes executado para verificar os comportamentos previstos do código.

Para investigar, usaremos **logs**, registros produzidos pela aplicação, e **eventos**, registros do Kubernetes sobre situações como agendamento, início de contêineres e falhas de probes. Publicar sem consultar esses sinais deixa dúvidas sobre o resultado da entrega.

5 | Preparar o computador

Ferramentas necessárias

Separe um computador por grupo. Faça a instalação antes da aula: a primeira execução precisa baixar pacotes e imagens e pode demorar mais que as etapas práticas.

Ferramenta	Por que vamos utilizá-la
Python 3.10+	Executar a aplicação, os testes e os scripts auxiliares. Não há bibliotecas adicionais a instalar com pip.
Docker	Construir a imagem e fornecer o ambiente usado pelo driver do Minikube.
Minikube	Criar o cluster Kubernetes local e executar kubectl.
Terminal	PowerShell no Windows; Bash no Ubuntu e no Arch.
Editor e navegador	Alterar YAML e consultar a página. Pode utilizar o editor que já conhece.

O Minikube exige pelo menos duas CPUs, 2 GB de memória livre e 20 GB de disco livre. Esses valores se referem aos recursos disponíveis para o ambiente. O Windows, o navegador e o editor também precisam de memória. Uma máquina com 8 GB de RAM ou mais costuma ser mais confortável para a aula. [9]

Os passos seguintes consideram **Windows 11 x86_64**, **Ubuntu 22.04 ou superior x86_64** e **Arch Linux x86_64**. Em máquinas ARM, utilize os instaladores da arquitetura correspondente. No Windows, a virtualização de hardware deve estar habilitada; Docker Desktop informa se esse recurso estiver indisponível.

Há diferenças de instalação e de terminal, mas a aplicação e os quatro arquivos YAML são os mesmos nos três sistemas. Nos roteiros, cada bloco informa em qual terminal deve ser executado. Não copie comandos Bash no PowerShell: variáveis, caminhos e continuação de linha têm sintaxes diferentes.

No Windows, a atividade será feita no **PowerShell nativo**. O Docker Desktop utilizará WSL 2 para executar contêineres Linux; isso não exige que você execute os comandos da aula dentro do Ubuntu do WSL. Para quem já utiliza Ubuntu no WSL, há uma orientação ao final do tutorial de Windows.

Não será necessário instalar Git ou utilizar repositórios remotos. O pacote de código será fornecido diretamente. A internet será utilizada nos downloads iniciais e na obtenção da imagem-base Python; as imagens produzidas pelo grupo ficarão locais.

6 | Instalação no Windows

WSL 2 e Docker Desktop

Comece pelo ambiente que executará os contêineres. Estes passos são para Windows 11. Se o Docker Desktop já funciona com contêineres Linux, confira os itens 3 e 4 e siga para a próxima página.

1. Habilite o WSL 2. Abra o menu Iniciar, procure PowerShell e escolha *Executar como administrador*. Execute:

```
wsl --install --no-distribution
wsl --update
wsl --set-default-version 2
```

Reinicie o computador se a instalação solicitar. A opção `-no-distribution` prepara o WSL sem instalar Ubuntu: neste caminho, usaremos o PowerShell e o Docker Desktop. [13]

2. Instale o Docker Desktop. Acesse `https://docs.docker.com/desktop/setup/install/windows-install/`, baixe o instalador Windows x86_64 e execute-o. Se o instalador apresentar a escolha, selecione o backend WSL 2. Ao terminar, abra o Docker Desktop e espere o mecanismo ficar em execução. [12]

3. Confira a configuração. Em *Settings* > *General*, verifique a opção *Use the WSL 2 based engine*, quando exibida. O Docker precisa utilizar contêineres Linux. Se o menu do aplicativo oferecer *Switch to Linux containers*, selecione essa opção. Não habilite o Kubernetes próprio do Docker Desktop: o cluster da aula será criado pelo Minikube.

4. Teste no PowerShell comum. Após reabrir o terminal, execute:

```
wsl --version
docker version
docker info
```

Em `docker version`, devem aparecer informações do cliente e do servidor. Se o cliente existir, mas o servidor não responder, verifique se o Docker Desktop está aberto e terminou de iniciar. `docker info` não deve terminar com erro de conexão ao mecanismo.

Se você já utiliza Ubuntu no WSL. É possível seguir os comandos Bash, mas mantenha todo o trabalho nesse ambiente. Habilite a integração da distribuição em *Settings* > *Resources* > *WSL Integration*; nela, instale Python, curl, unzip e o Minikube conforme Ubuntu. Não instale outro Docker Engine nessa distribuição para este roteiro: utilize a integração com o Docker Desktop. O perfil e os arquivos do caminho WSL são separados dos utilizados no PowerShell.

> O restante da instalação de Windows e os blocos “Windows (PowerShell)” da atividade seguem o caminho nativo. Não é preciso alternar entre terminais Windows e Linux durante a aula.

7 | Instalação no Windows

Python, Minikube e arquivos da aula

1. Instale Python. No PowerShell, tente `python -version`. Se já houver Python 3.10 ou superior funcionando, mantenha essa instalação. Caso contrário, acesse `https://www.python.org/downloads/windows/` e instale o Python Install Manager. Reabra o PowerShell e instale uma versão estável:

```
py install 3.14
python --version
```

O comando `py install` é do Install Manager. Se o `py` existente não reconhecer esse comando, ele pode ser o launcher antigo: conclua a instalação do Manager ou utilize o instalador tradicional disponibilizado pelo projeto. No instalador tradicional, habilite Python no PATH. Se `python` abrir a Microsoft Store, confira a instalação e os aliases de execução nas configurações do Windows. [14]

2. Instale Minikube. O instalador abaixo é distribuído pelo endereço oficial indicado na página do Minikube. Execute no PowerShell, em uma pasta onde possa salvar arquivos. O instalador poderá solicitar autorização de administrador.

```
$url = 'https://storage.googleapis.com/minikube/releases/latest/'
$url += 'minikube-installer.exe'
Invoke-WebRequest -Uri $url -OutFile '.\minikube-installer.exe'
Start-Process '.\minikube-installer.exe' -Wait
```

Conclua as telas do instalador e reabra o terminal para carregar o PATH. Confira `minikube version`. Se não for reconhecido, verifique se a pasta que contém `minikube.exe` foi incluída no PATH pelo instalador. [9]

3. Extraia o projeto. Considerando o ZIP na pasta Downloads, os comandos seguintes criam uma pasta de trabalho. Ajuste o caminho de origem se salvou o arquivo em outro lugar.

```
New-Item -ItemType Directory -Force "$HOME\aulas" | Out-Null
Unblock-File "$HOME\Downloads\devops_kubernetes.zip"
Expand-Archive "$HOME\Downloads\devops_kubernetes.zip" "$HOME\aulas"
Set-Location "$HOME\aulas\devops_kubernetes"
Get-ChildItem .\scripts\*.ps1 | Unblock-File
Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned
```

A política acima vale somente para o terminal atual. Em cada nova janela que executar scripts PowerShell, ajuste novamente o escopo Process, se necessário. Em computadores administrados pela instituição, uma política central pode exigir liberação pelo suporte.

4. Confirme as ferramentas.

```
python --version
docker info
minikube version
```

Agora o computador está pronto para iniciar o cluster. Não há dependências Python adicionais nem instalação separada de kubectl.

8 | Instalação no Ubuntu

Pacotes e permissão de acesso ao Docker

Execute esta página no terminal Bash de Ubuntu 22.04 ou superior. O tutorial utiliza o pacote Docker mantido pela distribuição. Se você já instalou Docker por outro método e `docker info` funciona, não instale `docker.io` por cima dele.

1. Instale os pacotes necessários.

```
sudo apt update
sudo apt install -y docker.io python3 curl unzip
sudo systemctl enable --now docker
sudo usermod -aG docker "$USER"
```

`apt` instala os pacotes; `systemctl` inicia o serviço e o habilita no próximo boot; `usermod` adiciona sua conta ao grupo Docker. Esse grupo permite controlar o mecanismo Docker e tem amplo acesso ao sistema. Saia da sessão e entre novamente para aplicar a nova associação. Depois, teste `docker info` *sem sudo*. [15]

2. Instale Minikube. Confira a arquitetura. O exemplo abaixo usa o pacote amd64, correspondente a x86_64.

```
uname -m
curl -fLO https://storage.googleapis.com/minikube/releases/latest/minikube_latest_amd64.deb
sudo dpkg -i minikube_latest_amd64.deb
minikube version
```

Se `uname -m` mostrar `aarch64`, utilize o pacote arm64 indicado na documentação oficial, em vez de amd64. O pacote Debian fornecido pelo projeto evita a necessidade de configurar outro repositório. [9]

3. Extraia o material. Considerando o ZIP em `/Downloads`, crie a pasta e entre no projeto:

```
mkdir -p ~/aulas
unzip ~/Downloads/devops_kubernetes.zip -d ~/aulas
cd ~/aulas/devops_kubernetes
chmod +x scripts/kube.sh scripts/pipeline_local.sh
python3 --version
docker info
minikube version
```

`chmod +x` permite executar os scripts Bash pelo caminho do arquivo. Se o ZIP estiver em outra pasta, modifique apenas o caminho informado ao unzip.

O cluster será iniciado com seu usuário normal. Se o Minikube informar que não consegue acessar o Docker, resolva primeiro o serviço ou a associação ao grupo; não transforme a execução do laboratório em uma sessão root.

Ubuntu dentro do WSL. Se escolheu esse caminho no Windows, instale apenas `python3 curl unzip` na primeira linha de instalação. Pule `systemctl` e `usermod`: o Docker será fornecido pela integração do Desktop. O Minikube e o código, nesse caso, devem permanecer dentro do mesmo ambiente WSL.

9 | Instalação no Arch

Preparação pelo gerenciador de pacotes

No Arch, utilizaremos os pacotes da distribuição. Abra o terminal e execute:

```
sudo pacman -Syu --needed docker minikube python curl unzip
sudo systemctl enable --now docker
sudo usermod -aG docker "$USER"
```

`-Syu` atualiza o sistema e sincroniza os pacotes; `-needed` evita reinstalar pacotes que já atendem à solicitação. O nome do pacote Python no Arch é `python`; no roteiro, usaremos o comando `python3`. Confira se ele está disponível após a instalação. [15]

Saia da sessão e entre novamente para que o grupo Docker seja aplicado. Em seguida, execute sem sudo:

```
python3 --version
docker version
docker info
minikube version
```

O cliente e o servidor Docker devem responder. Caso o serviço não esteja funcionando, consulte seu estado:

```
systemctl status docker --no-pager
```

Depois de baixar o pacote da aula, extraia-o e entre na pasta, como no Ubuntu:

```
mkdir -p ~/aulas
unzip ~/Downloads/devops_kubernetes.zip -d ~/aulas
cd ~/aulas/devops_kubernetes
chmod +x scripts/kube.sh scripts/pipeline_local.sh
```

Se você já possui Docker funcionando, preserve a instalação e confira os comandos antes de fazer alterações. O que precisamos para a aula é um mecanismo acessível pelo usuário, Python e Minikube disponíveis no terminal.

O que muda entre Ubuntu e Arch na prática?

Muda o gerenciador de pacotes e o nome de alguns pacotes. Depois da instalação, os comandos Bash do laboratório são iguais: `python3`, `docker`, `minikube` e o script `kube.sh`. Os objetos do Kubernetes não dependem do gerenciador de pacotes utilizado na máquina.

A versão mostrada pelas ferramentas pode variar entre distribuições. Registre essas versões no relatório. Se um resultado divergir, essa informação ajuda a reproduzir e investigar o que ocorreu.

10 | Primeiro uso

Iniciar o cluster e conferir o contexto

Todos os comandos da atividade serão executados na pasta `devops_kubernetes`, onde ficam o Dockerfile e as pastas `app`, `k8s` e `scripts`. Quando abrir um segundo terminal, entre novamente nessa pasta.

Inicie o perfil `devops-k8s`. Um perfil identifica uma instalação local do Minikube. Os parâmetros abaixo reservam duas CPUs e 2048 MB para esse cluster; o driver Docker informa como o nó será criado.

Ubuntu e Arch (Bash)

```
mkdir -p build evidencias
minikube -p devops-k8s start --driver=docker --cpus=2 --memory=2048
./scripts/kube.sh get nodes
python3 -m unittest discover -s tests -v
```

Windows (PowerShell)

```
New-Item -ItemType Directory -Force build,evidencias | Out-Null
minikube -p devops-k8s start --driver=docker --cpus=2 --memory=2048
.\scripts\kube.ps1 get nodes
python -m unittest discover -s tests -v
```

O nó deve chegar ao estado `Ready`. A primeira inicialização pode demorar porque precisa baixar imagens. Os testes da aplicação devem concluir com oito testes aprovados. Eles podem ser executados antes de criar o cluster, pois usam servidores locais temporários.

Os scripts `kube.sh` e `kube.ps1` não criam outra ferramenta Kubernetes: apenas chamam o kubectl do Minikube com contexto e namespace fixados. Assim, `get pods` sempre consulta os Pods do exercício. Antes da publicação, o namespace ainda não existe; ele será criado pelo manifesto fornecido.

Por que uma imagem local precisa ser carregada?

Construir a imagem no Docker do computador não garante que o nó Kubernetes a encontre no seu próprio armazenamento. Por isso usamos `minikube image load`, com o perfil correto. A política `imagePullPolicy: Never` faz o contêiner utilizar somente uma imagem disponível no nó; se ela estiver ausente, ele não inicia. [7,10]

O arquivo `k8s/deployment.yaml` é o modelo-base. `render.py` gera `build/deployment.yaml` com a versão e as réplicas escolhidas. Utilize `-output` para salvar: o script grava UTF-8 tanto no Windows quanto no Linux, evitando diferenças de codificação causadas pelo redirecionamento de saída do PowerShell antigo.

11 | Leitura dos arquivos

Configuração e recursos da aplicação

Antes de executar a atividade, confira os seguintes arquivos:

Arquivo	O que contém
`app/main.py`	Servidor HTTP e comportamento de cada rota.
`tests/test_app.py`	Testes dos comportamentos previstos.
`Dockerfile`	Construção da imagem.
`k8s/namespace.yaml`	Organização dos objetos em aula-k8s.
`k8s/configmap.yaml`	Mensagem recebida pela aplicação.
`k8s/service.yaml`	Selector e portas de acesso.
`k8s/deployment.yaml`	Modelo com imagem, réplicas, probes e recursos.
`scripts/`	Comandos auxiliares e pipeline local.

A aplicação mostra a versão, a mensagem e o nome do Pod que respondeu. A rota `/api/status` apresenta as mesmas informações como JSON. `/healthz` informa que o servidor consegue responder; `/readyz` verifica a condição de prontidão utilizada pelo exercício. Na versão `quebrada`, apenas a prontidão falha de propósito.

No Deployment, **requests** indicam recursos considerados para agendar o contêiner; **limits** restringem seu consumo. Pedimos 100m de CPU e 32Mi de memória, com limites de 500m e 96Mi. `100m` equivale a 0,1 CPU e `Mi` é uma unidade binária de memória. São valores escolhidos para este servidor pequeno, não uma receita para outras aplicações. [8]

As linhas `maxSurge: 1` e `maxUnavailable: 0` permitem uma instância adicional na atualização e procuram preservar a quantidade disponível. `progressDeadlineSeconds: 90` define o prazo para considerar que uma atualização não progride; isso não executa rollback automaticamente.

Do exercício para um ambiente de trabalho

Os conceitos são os mesmos, mas nossa configuração é reduzida. Uma aplicação real pode exigir armazenamento persistente, autenticação, acesso externo, mais de um nó e acompanhamento contínuo. Aqui, manteremos o problema pequeno para que seja possível observar cada decisão.

Na atividade, leia o efeito esperado antes de executar. Quando ele não ocorrer, utilize os comandos de observação para descobrir em que etapa o processo parou. Essa investigação faz parte do trabalho, e não é apenas um obstáculo para terminar o roteiro.

Referências

Documentação utilizada

As definições foram adaptadas da documentação oficial. O código, as comparações e o roteiro foram preparados para esta atividade. Os exemplos profissionais relacionam os conceitos da aula a situações de publicação e manutenção de aplicações.

[1] Kubernetes. Cluster Architecture.
https://kubernetes.io/docs/concepts/architecture/

[2] Kubernetes. Pods.
https://kubernetes.io/docs/concepts/workloads/pods/

[3] Kubernetes. Deployments.
https://kubernetes.io/docs/concepts/workloads/controllers/deployment/

[4] Kubernetes. Service.
https://kubernetes.io/docs/concepts/services-networking/service/

[5] Kubernetes. ConfigMaps.
https://kubernetes.io/docs/concepts/configuration/configmap/

[6] Kubernetes. Liveness, Readiness and Startup Probes.
https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/

[7] Kubernetes. Images.
https://kubernetes.io/docs/concepts/containers/images/

[8] Kubernetes. Resource Management for Pods and Containers.
https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/

[9] Minikube. Start.
https://minikube.sigs.k8s.io/docs/start/

[10] Minikube. Pushing images.
https://minikube.sigs.k8s.io/docs/handbook/pushing/

Para consultar o funcionamento específico do encaminhamento de portas:

https://kubernetes.io/docs/tasks/access-application-cluster/port-forward-access-application-cluster/

Referências

Instalação e uso profissional

[11] Docker. What is a container?
https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/

[12] Docker. Install Docker Desktop on Windows; WSL 2 backend.
https://docs.docker.com/desktop/setup/install/windows-install/

[13] Microsoft. Instalar WSL e comandos básicos.
https://learn.microsoft.com/pt-br/windows/wsl/basic-commands

[14] Python. Using Python on Windows.
https://docs.python.org/3/using/windows.html

[15] Ubuntu Packages e Arch Linux Packages. Pacotes da instalação.
https://packages.ubuntu.com/noble/docker.io

[16] AWS, Microsoft e Google. Serviços Kubernetes gerenciados.
https://docs.aws.amazon.com/eks/latest/userguide/what-is-eks.html

[17] AWS. What is DevOps?
https://aws.amazon.com/devops/what-is-devops/

Complementos das referências [12], [13], [15] e [16]:

- Docker e WSL:
https://docs.docker.com/desktop/features/wsl/
- Instalação do WSL:
https://learn.microsoft.com/pt-br/windows/wsl/install/
- Pacote Minikube no Arch:
https://archlinux.org/packages/extra/x86_64/minikube/
- Azure Kubernetes Service:
https://learn.microsoft.com/pt-br/azure/aks/what-is-aks
- Google Kubernetes Engine:
https://cloud.google.com/kubernetes-engine/docs/concepts/kubernetes-engine-overview

Consulta em 09/10/2026. As telas de instalação e as versões disponíveis podem mudar. Quando houver divergência de interface, utilize a documentação da ferramenta instalada.